Monitoreo Inteligente del Secado de Café en Marquesinas: Solución Cloud-IoT de Extremo a Extremo sobre Infraestructura Propia

Ing. Robin Mauricio Dussán Medina
Corporación Universitaria del Huila (CORHUILA)
Integración Cloud en IoT [G1-2026B]
Actividad de aprendizaje 3 · Proyecto integrador (Unidad 3)
Docente: Jhon Ríos
2 de noviembre de 2026

Monitoreo Inteligente del Secado de Café en Marquesinas

Este informe presenta la solución integradora del módulo: un sistema Cloud-IoT completo que monitorea en tiempo real las marquesinas de secado de café pergamino, alerta cuando las condiciones se salen de rango, detecta anomalías y conserva un histórico apto para entrenar, más adelante, un modelo que estime el tiempo restante de secado. A diferencia de lo sugerido en el enunciado, no se usó un broker comercial (HiveMQ Cloud, AWS IoT Core o Azure IoT Hub): toda la plataforma se construyó sobre infraestructura propia —un servidor virtual privado (VPS) y una Raspberry Pi como nodo fog—, lo que obligó a resolver explícitamente la seguridad, el despliegue y el monitoreo que un proveedor gestionado ocultaría. El código está en el repositorio público https://github.com/Robindussan/a3-cafe-cloud-iot y el sistema funciona en https://panel.agrotecnet.cloud.

Caso y Requisitos

Una asociación de caficultores del sur del Huila seca su café pergamino en marquesinas (secadores solares). Si la temperatura o la humedad relativa dentro de la marquesina no se controlan, el grano puede sobresecarse o desarrollar hongos y perder calidad de taza; el objetivo de humedad del grano seco es de 10 % a 12 % (Roa et al., 1999). La asociación quiere monitorear 20 marquesinas, recibir alertas y guardar un histórico; la conectividad en las fincas es intermitente y el presupuesto es limitado.

Para el prototipo se usó un único montaje físico (ESP32-C3 con DHT22, LDR, MQ-7 y reloj DS3231) que se presenta ante el broker como **tres marquesinas** independientes (`marquesina-01` a `-03`): tres clientes MQTT con su propia conexión TLS, usuario, clave, última voluntad y tópicos. Las marquesinas 02 y 03 aplican un desfase fijo y un ruido menor que la banda muerta sobre la lectura real, para que sus series no sean idénticas. La Tabla 1 resume cómo se cumple cada requisito mínimo.

Tabla 1
Requisitos de la solución y su implementación

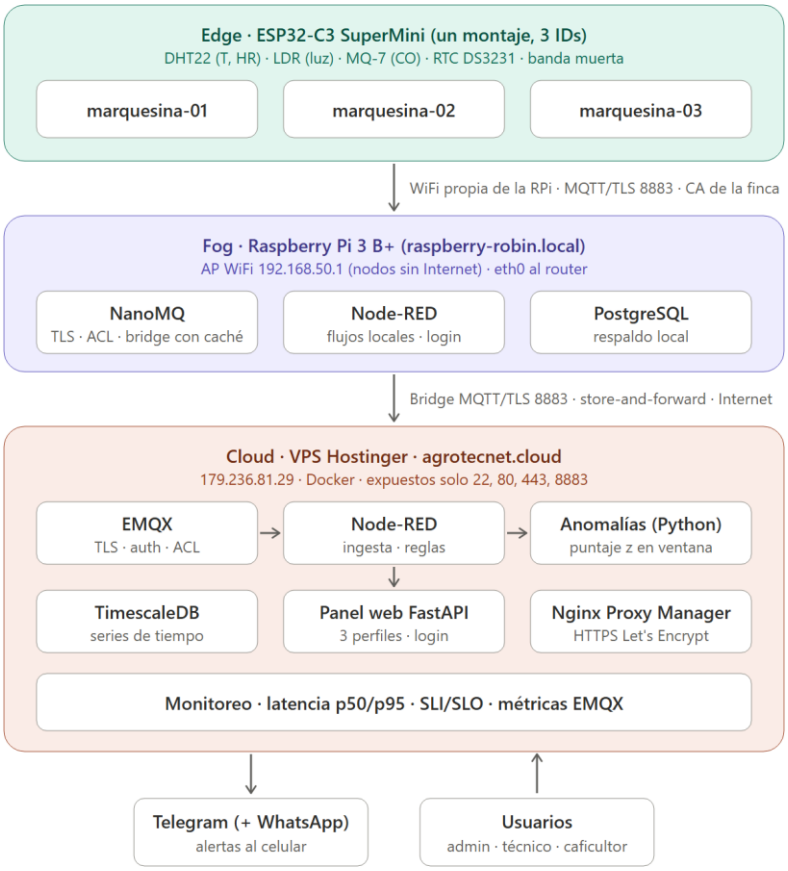
#	Requisito	Implementación	Evidencia
1	≥ 3 dispositivos, ≥ 2 variables, JSON con marca de tiempo	ESP32-C3 con 3 IDs; JSON {id, ts (ms UTC), seq, temp, hr, lux, co, rssi, m}; hora por NTP del fog y DS3231	Registro serie; tabla telemetria
2	MQTT sobre TLS con autenticación hacia la nube	Nodo → NanoMQ (TLS, CA de la finca); NanoMQ → EMQX (bridge TLS, Let's Encrypt); usuario/clave y ACL en ambos brokers	Tabla 4, Tabla 5
3	Base de series de tiempo	TimescaleDB 2.30 (hypertables, compresión a 7 d, retención 365 d)	Esquema SQL del repositorio
4	Tablero con estado e histórico	Panel web propio con 3 perfiles y gemelo digital WebGL	Figuras 2, 3 y 5
5	≥ 1 regla de alerta por canal verificable	Node-RED: umbrales editables, antirebote; Telegram, tópico cafe/alertas y tabla alertas	Tabla 9, Figura 4
6	Detección de anomalías que publique resultados	Servicio Python: puntaje z en ventana móvil → cafe/<id>/anomalia y tabla anomalias	Tabla 9, Figura 5
7	Optimización medida antes/después	Banda muerta en el nodo; 60 min sin y 60 min con	Tabla 6
8	Latencia p50/p95 y dos SLI con SLO	latencia_ms por lectura; SLI de frescura y de disponibilidad	Tabla 8

Arquitectura Final y Decisiones de Diseño

La Figura 1 presenta la arquitectura en tres niveles de cómputo (edge, fog y nube) más el nivel de usuarios. El principio rector fue procesar cada dato donde su latencia, su volumen y la conectividad lo justifican (Shi et al., 2016; Yousefpour et al., 2019): el nodo decide **cuándo** vale la pena transmitir, el fog garantiza que nada se pierda aunque se caiga Internet, y la nube concentra lo que necesita varias marquesinas o mucho histórico.

Figura 1

Arquitectura cloud–fog–edge de la solución



Nota. Elaboración propia. Los nodos no tienen salida a Internet: solo hablan con la Raspberry por MQTT/TLS y toman de ella la hora (NTP). Únicos puertos expuestos del VPS: 22, 80, 443 y 8883.

Edge: ESP32-C3

El firmware (versión 3.0.4) lee los sensores cada 10 s, sincroniza el reloj con el NTP de la Raspberry y guarda la hora en el DS3231 en UTC; no publica hasta tener hora confiable. Aplica **banda muerta** por marquesina (publica si la temperatura cambia 0,3 °C, la humedad 1 %, la luz 15 % o el CO 5 ppm, o cada 5 min como latido) y acepta comandos remotos (`leer_ahora`, `banda`, `latido`, `periodo`, `calibrar_mq7`). La telemetría usa QoS 0 en el tramo local y el bridge la sube con QoS 1; el estado se publica retenido y con

última voluntad, de modo que la nube sabe cuándo un nodo se desconecta (Banks et al., 2019). Las tres conexiones TLS simultáneas consumen unos 42 KB de memoria cada una y dejan 75 KB libres, por lo que no fue necesario el plan alternativo de una sola conexión.

Fog: Raspberry Pi 3 B+

La Raspberry crea su propia red WiFi (AgroTecnet-Fog, WPA2) para los nodos y sale a Internet por cable. Un filtro nftables impide que los nodos alcancen Internet, de modo que un nodo comprometido no puede atacar hacia afuera. NanoMQ 0.25.7 recibe a los nodos por TLS con un certificado emitido por una CA propia de la finca, aplica usuarios y ACL por nodo, y reenvía a la nube por un bridge TLS con caché en disco (*store-and-forward*; EMQ Technologies, 2026b). Se usó la imagen `-full`, porque la imagen estándar de NanoMQ no incluye TLS.

Nube: VPS

En un VPS Hostinger KVM 2 (2 vCPU, 8 GB) corren, en contenedores Docker de una red interna, EMQX 6.3.1 como broker de la nube (EMQ Technologies, 2026a), Node-RED 5.0.8 para ingesta y alertas, TimescaleDB 2.30 para las series (Timescale, 2026), un servicio Python de anomalías y el panel web en FastAPI; Nginx Proxy Manager publica los paneles con HTTPS. La Tabla 2 resume las decisiones de diseño más relevantes.

Tabla 2

Decisiones de diseño y su justificación

Decisión	Alternativa descartada	Justificación
Infraestructura propia (VPS + Raspberry)	HiveMQ Cloud / AWS IoT Core	Aprendizaje de toda la pila; costo fijo bajo (≈ COP 25.000/mes); sin dependencia de cuotas del proveedor
NanoMQ en el fog con bridge y caché	Nodos directos a la nube	Conectividad intermitente: el fog guarda y reenvía; los nodos quedan aislados de Internet
CA propia para el tramo local	Certificado público en la finca	La red local no tiene dominio público; la CA de la finca permite verificar el broker por nombre (fog.finca)
TimescaleDB	InfluxDB	SQL estándar para el panel y el dataset; compresión nativa; percentiles para p50/p95 en la misma consulta
Panel propio (FastAPI)	Node-RED Dashboard / Grafana	Perfiles con autorización por marquesina en el servidor; gemelo digital; descarga del dataset
Telegram como canal principal	WhatsApp (CallMeBot)	API oficial y confirmación de entrega; la cuenta gratuita de CallMeBot quedó pausada y se dejó como canal secundario

El panel (Figura 2) ofrece tres perfiles: el **administrador** gestiona usuarios, umbrales y descarga el dataset; el **técnico** ve todas las marquesinas, la latencia y los SLI; el **caficultor** solo ve sus marquesinas. La autorización se verifica en el servidor en cada página y en cada consulta de la API: una marquesina ajena responde 404. La Figura 3 muestra el **gemelo digital** en WebGL de una marquesina de 8 × 4 m en alambrado: el color y la velocidad del aire caliente siguen a la temperatura, el color del café sigue a la humedad, el sol a la radiación, el humo de la estufa al CO y el pulso del nodo a su conexión.

Figura 2

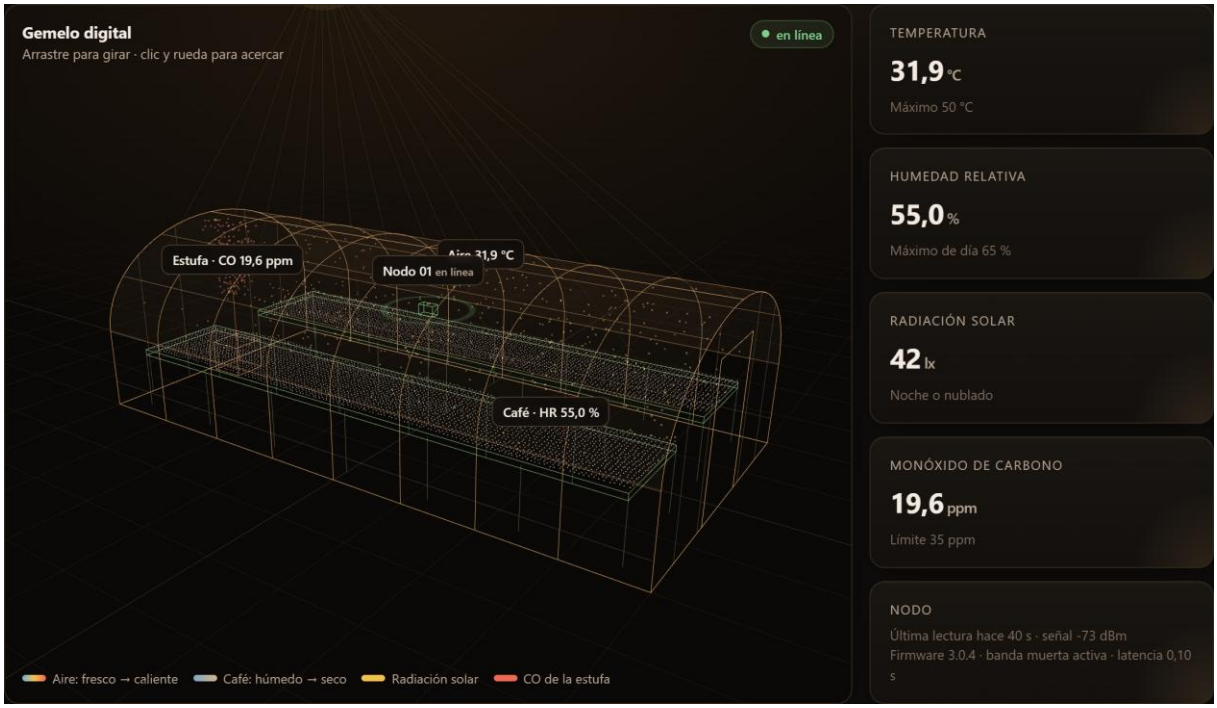
Resumen del panel web con indicadores y objetivos de servicio



Nota. Captura del 2 de noviembre de 2026, perfil administrador. Incluye marquesinas en línea, alertas, latencia p50/p95 y SLI/SLO.

Figura 3

Gemelo digital en WebGL de la marquesina 1



Nota. Escena Three.js que reacciona a los datos en vivo; las etiquetas 3D muestran el valor de cada variable.

Seguridad

Modelo de Amenazas

Se actualizó el modelo STRIDE de la Actividad 2 (Shostack, 2014) con los componentes efectivamente desplegados. La Tabla 3 presenta las amenazas principales, el control aplicado y el riesgo residual.

Tabla 3

Modelo de amenazas STRIDE actualizado

Amenaza (STRIDE)	Activo	Control implementado	Riesgo residual
Suplantación (S): un equipo se hace pasar por una marquesina	NanoMQ / EMQX	Usuario y clave por nodo; ACL que solo permite cafe/<su id>/#; validación id = tópico en Node-RED	Claves en el firmware; mitigable con certificado de cliente (mTLS)
Suplantación (S): broker falso en la finca	Nodos	El nodo verifica el certificado de fog.finca firmado por la CA de la finca	Bajo
Manipulación (T) y divulgación (I) en tránsito	MQTT	TLS 1.3/1.2 en ambos tramos; puerto 1883 cerrado en la nube	Bajo
Repudio (R) de cambios de umbrales o alertas	Panel	Cada umbral guarda quién y cuándo lo cambió; alertas reconocidas con el usuario	Medio: sin registro de auditoría completo
Divulgación (I): acceso a datos ajenos	Panel / BD	Perfiles con autorización en el servidor; roles de BD de mínimo privilegio; BD sin puerto público	Bajo

Amenaza (STRIDE)	Activo	Control implementado	Riesgo residual
Denegación de servicio (D)	VPS	ufw con 4 puertos; fail2ban en SSH; antirebote de alertas; bloqueo de login tras 5 intentos	Medio: un solo VPS (sin alta disponibilidad)
Elevación de privilegios (E)	Contenedores	Contenedores sin puertos publicados, panel de solo lectura y usuario sin privilegios; editor de Node-RED con login	Medio: SSH con clave aún habilitado; sudo sin clave en la Raspberry

Autenticación, Autorización y Gestión de Claves

Cada actor tiene su propia identidad y solo los permisos que necesita (Tabla 4). En EMQX las claves se guardan como hash SHA-256 con sal; en el panel, con bcrypt de costo 12, y la sesión viaja en una cookie firmada, HttpOnly y Secure, con token CSRF en los formularios. Ninguna credencial está en el repositorio: el código solo contiene archivos `*.example`, los secretos viven en directorios con permisos 600 en cada equipo y el pipeline de integración continua ejecuta gitleaks en cada envío. Los certificados públicos se renuevan solos (Let's Encrypt), y un script sincroniza diariamente el certificado de `mqtt.agrotecnet.cloud` hacia EMQX; las claves privadas de la CA de la finca permanecen solo en la Raspberry, siguiendo las recomendaciones de gestión de claves de Barker (2020).

Tabla 4

Identidades y permisos de mínimo privilegio

Identidad	Dónde	Permisos
marquesina-01 … 03	NanoMQ (fog)	Publicar cafe/<id>/telemetria y estado; suscribirse a cafe/<id>/cmd
gateway-rpi	EMQX	Publicar telemetria, estado y agregado; recibir cmd
svc-nodered	EMQX	Leer cafe/#; publicar cafe/alertas y cafe/+/cmd
svc-anomalias	EMQX	Leer telemetría; publicar cafe/+/anomalia
app_nodered / app_panel / app_anomalias	TimescaleDB	Solo las tablas y operaciones que usa cada servicio
admin / técnico / caficultor	Panel	Administración completa / monitoreo sin administración / solo sus marquesinas

Evidencia de TLS

La Tabla 5 resume las pruebas realizadas. Para el tramo hacia la nube se capturó el tráfico del puerto 8883 en el VPS con tcpdump mientras se publicaba un mensaje que contenía un texto reconocible; el análisis con tshark muestra el saludo TLS 1.3 y que ese texto no aparece en la captura (Rescorla, 2018).

Tabla 5

Pruebas de seguridad realizadas

Prueba	Resultado
Saludo TLS cliente → mqtt.agrotecnet.cloud:8883	TLS 1.3, suite TLS_AES_256_GCM_SHA384, SNI mqtt.agrotecnet.cloud, certificado Let's Encrypt válido
¿Se ve el contenido MQTT en la captura?	No: 45 registros cifrados, 0 paquetes decodificables como MQTT, 0 apariciones del texto marcado

Prueba	Resultado
Bridge Raspberry → nube	TLS 1.2 (mbedTLS de NanoMQ) con verificación del certificado
Conexión al puerto 1883 sin cifrar	Sin respuesta (bloqueado por el firewall y sin puerto publicado)
Usuario anónimo / clave errada	Rechazados: Not authorized / Bad user name or password
Publicar fuera de la ACL	PUBACK 135 (Not authorized) en EMQX; desconexión en NanoMQ
Caficultor consulta una marquesina ajena	HTTP 404 en la página y en la API

Optimización: Técnica Aplicada y Resultados

La técnica implementada es la **banda muerta** (envío por excepción): el nodo solo publica cuando una variable cambia más que su umbral respecto al último valor enviado, o cuando vence un latido de 5 min que permite distinguir "todo estable" de "nodo caído". Para medirla, un programa en la nube (usuario MQTT `svc-monitor`) desactivó la banda en las tres marquesinas durante 60 min, la reactivó otros 60 min y, en cada fase, contó los mensajes que llegaron a EMQX, los bytes de carga útil, una estimación de bytes en el aire (cabecera MQTT y registro TLS) y la latencia. La Tabla 6 muestra los resultados.

Tabla 6

Resultados antes y después de la banda muerta (3 marquesinas)

Métrica	Sin banda muerta	Con banda muerta	Cambio
Mensajes por hora	1.080	120	−88,9 %
Bytes en el aire por hora (estimado)	184.631	20.683	−88,8 %
Mensajes por marquesina por hora	360	40	
Latencia nodo → nube p50 / p95 (ms)	36 / 1.383	58 / 87	
Motivo de los mensajes	periódico (p)	114 por cambio (c) y 6 latidos (l)	

Nota. Medición del 2 de noviembre de 2026, de 16:01 a 17:01 (fase sin banda) y de 17:01 a 18:01 (fase con banda), hora de Colombia. Bytes en el aire = tópico + carga + 4 B de cabecera MQTT + 29 B de registro TLS por mensaje.

Con la banda muerta cada marquesina pasó de 360 a unos 40 mensajes por hora; 114 de los 120 mensajes de la fase con banda se enviaron por un cambio real y solo 6 fueron latidos, de modo que no se perdió información relevante: los cambios bruscos (que son los que disparan alertas) se publican de inmediato. La latencia p95 bajó de 1.383 ms a 87 ms porque, con menos mensajes, se redujeron las colas en el WiFi y en el bridge; la mediana apenas cambió (36 ms frente a 58 ms). El ahorro se traslada a la nube: 89 % menos filas en la base y menos trabajo de ingesta, alertas y detección de anomalías. La reducción depende de la variabilidad: en horas de sol cambiante habrá más envíos que en esta medición en interior.

La conectividad intermitente se probó cortando el acceso de la Raspberry al VPS durante 90 s mientras una marquesina publicaba cada 10 s (Tabla 7). La caché del bridge guardó los mensajes y los reenvió al volver la conexión; uno se perdió en la ventana en que el bridge aún no detectaba la caída (keepalive de 60 s), por lo que el keepalive se redujo a 20 s. Además, un índice único (marquesina, ts, seq) evita duplicados cuando el bridge reenvía.

Tabla 7
Prueba de store-and-forward con corte de Internet de 90 s

Mensajes	Comportamiento	Retraso al llegar
21–22	antes del corte	0–1 s
23	enviado justo al caer el enlace (perdido)	—
24–31	guardados en la caché del bridge y reenviados al volver	46–81 s
32 en adelante	después del corte, intercalados con el reenvío	1 s

Plan de Despliegue

Infraestructura como código. Todo lo que corre está descrito en archivos versionados: los stacks de Docker de la nube y del fog con etiquetas de imagen exactas (sin `latest`), la configuración y ACL de los brokers, el esquema y los roles de la base, los flujos de Node-RED como código con su instalador, la red del fog y la PKI de la finca. Replicar una finca consiste en instalar el sistema operativo, copiar la carpeta `fog/`, ejecutar los scripts de PKI y de red, y levantar los stacks.

CI/CD. Un flujo de GitHub Actions revisa que no haya credenciales (gitleaks), valida la sintaxis de Python y JavaScript, compila el firmware del ESP32 con datos de ejemplo y construye las imágenes del panel y del servicio de anomalías; en la rama principal las publica en GHCR y, con aprobación manual, las despliega en el VPS por SSH con un script que verifica que el panel responda.

Actualización OTA por etapas. El ESP32-C3 tiene dos ranuras de aplicación: se escribe la inactiva y, si el nuevo firmware no confirma su arranque conectándose al broker, el gestor de arranque vuelve al anterior. La orden llega por `cafe/<id>/cmd` con la URL HTTPS y el SHA-256 del binario, y se despliega en anillos: una marquesina canario (24 h), un piloto del 10 % (48 h) y luego lotes del 20 %; se avanza solo si los SLI se mantienen y se detiene si se agota el presupuesto de error.

Despliegue de modelos. El detector de anomalías es un contenedor versionado cuyos parámetros quedan registrados en cada resultado (campo `metodo`). Durante las pruebas, la versión 1.0, que excluía de la ventana de referencia las lecturas anómalas, marcó como anomalía cada lectura de una deriva lenta del MQ-7 (695 anomalías de CO en una hora); la versión 1.1 incorpora todas las lecturas a la ventana y se desplegó cambiando la etiqueta de la imagen. Para modelos futuros (p. ej. Isolation Forest, Liu et al., 2008, o un regresor de tiempo de secado entrenado con el dataset del panel) se propone un despliegue en sombra de 7 días antes de reemplazar al vigente.

Plan de Monitoreo

Cada lectura guarda la hora del nodo (`ts`) y la hora en que Node-RED la escribe en la base (`ts_ingesta`); su diferencia es la latencia de extremo a extremo, que incluye el WiFi, el fog, el bridge, EMQX y Node-RED. Con ella se definieron dos indicadores de nivel de servicio siguiendo a Beyer et al. (2016): la **frescura** (qué fracción de lecturas llega a tiempo) y la **disponibilidad de datos** (qué fracción de ventanas de 10 min tiene al menos una lectura, el doble del latido). La Tabla 8 los muestra con su objetivo y el presupuesto de error restante; el panel los presenta al administrador y al técnico.

Tabla 8

Latencia de extremo a extremo y objetivos de servicio (SLI/SLO)

Indicador	Definición	Medido (14:19–18:22)	SLO	Presupuesto restante
Latencia p50 / p95	ts_ingesta − ts de cada lectura	63 ms / 1.234 ms (1.419 lecturas)	—	—
SLI-1 Frescura	% de lecturas en la base en < 2 s	97,7 %	95 % (7 días)	54,8 %
SLI-2 Disponibilidad	% de ventanas de 10 min con datos, por marquesina	100 % (01, 02 y 03; 25/25 ventanas)	99 % (mensual)	100 %

Las alertas cubren tres familias: reglas de umbral (temperatura > 50 °C por riesgo de sobresecado, humedad > 65 % de día por secado lento y riesgo de hongos, CO > 35 ppm por la estufa de apoyo), desconexión o ausencia de datos de una marquesina y anomalías estadísticas. Todas pasan por un antirebote de 10 min y se publican en `cafe/alertas`, se guardan en la base y se envían a Telegram; la tabla de alertas registra si cada canal confirmó la entrega. La Tabla 9 resume las pruebas controladas, la Figura 4 muestra las alertas recibidas en el celular y la Figura 5 el histórico con los marcadores.

Tabla 9

Pruebas de alertas y de detección de anomalías

Prueba	Resultado
Umbral de CO reducido a 5 ppm (marquesina-01)	Alerta co_alto generada y entregada por Telegram (telegram = true)
Reinicio del ESP32	Alertas offline por última voluntad MQTT en las 3 marquesinas
Aliento sobre el DHT22 (16:06)	Temperatura +2,6 °C; puntaje z 6,3–6,7; 3 alertas anomalia_temp entregadas por Telegram
Deriva lenta del MQ-7 (v1.0 del detector)	Falsos positivos repetidos → corregido en la versión 1.1
Corte espontáneo del WiFi del nodo (18:30)	Alertas offline de las 3 marquesinas en Telegram; reconexión automática en unos 40 s

Figura 4

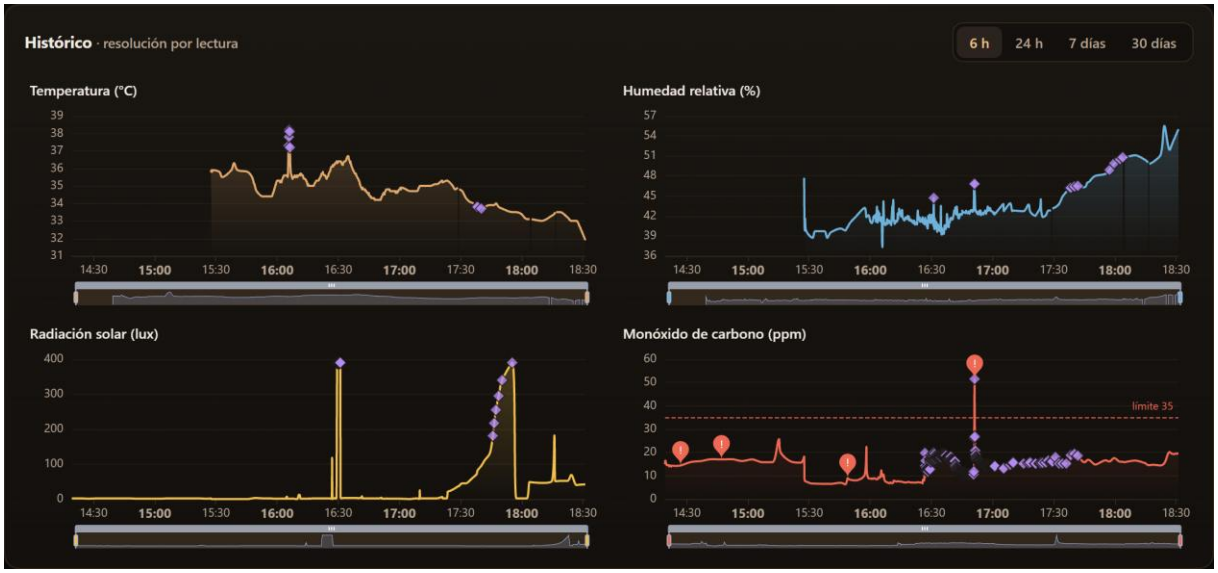
Alertas recibidas en Telegram (bot @Agrotecnetbot)



Nota. Captura del celular del autor, 2 de noviembre de 2026. Se ven las alertas de desconexión por última voluntad MQTT y las de anomalía con su puntaje z y el valor de referencia.

Figura 5

Histórico de 6 h con umbrales, alertas y anomalías



Nota. Los rombos violeta son anomalías del servicio de puntaje z; los marcadores rojos, alertas de umbral. Se observa el pico de temperatura de la prueba controlada y la deriva del MQ-7.

Estimación de Costos Mensuales

La Tabla 10 estima los costos para 20 y para 2.000 marquesinas con los volúmenes medidos con banda muerta (40 mensajes por marquesina por hora). Se supone una Raspberry por finca con tres marquesinas en promedio y un plan de datos móviles por finca; los precios de hardware y datos son referencias del mercado colombiano en 2026 y deben confirmarse con cotizaciones.

Tabla 10

Costos mensuales estimados (COP)

Concepto	20 marquesinas	2.000 marquesinas	Supuesto
Fincas (1 Raspberry cada una)	7	667	3 marquesinas por finca en promedio
Mensajes al mes (con banda muerta)	576.000	57.600.000	40 mensajes/marquesina/h medidos
Mensajes al mes (sin banda muerta)	5.184.000	518.400.000	360 mensajes/marquesina/h medidos
Datos en la BD al mes, sin comprimir	0,07 GB	6,9 GB	~120 B por fila; TimescaleDB comprime ~10× a los 7 días
Nube (VPS)	$ 25.000	$ 100.000	Real: KVM 2 por COP 600.000/24 meses. 2.000: KVM 4 + VPS de respaldo + copias externas (estimado)
Dominio, certificados y Telegram	$ 0	$ 0	Dominio incluido en el plan; Let's Encrypt y Telegram gratuitos
Datos móviles	$ 70.000	$ 6.670.000	Plan prepago de 1 GB por finca (COP 10.000); consumo estimado 44 MB/mes por finca (248 MB sin banda muerta)

Concepto	20 marquesinas	2.000 marquesinas	Supuesto
Hardware amortizado (36 meses)	$ 106.667	$ 10.373.333	Nodo COP 80.000; Raspberry con fuente y SD COP 320.000 por finca
Total mensual	$ 201.667	$ 17.143.333	
Total por marquesina	$ 10.083	$ 8.572	
Solo nube por marquesina	$ 1.250	$ 50	

Nota. Valores en pesos colombianos por mes. El precio del VPS es el pagado por el autor; los demás son supuestos de referencia del mercado colombiano en 2026. No incluye mano de obra de instalación ni mantenimiento.

Con 20 marquesinas el VPS actual sobra: recibe menos de un mensaje cada 5 s y la base crece 0,07 GB al mes. A 2.000 marquesinas el flujo sigue siendo bajo para EMQX y TimescaleDB (22 mensajes/s y 667 conexiones de bridge), pero se recomienda un VPS mayor y uno de respaldo por disponibilidad y copias de seguridad, no por capacidad; además, EMQX Community no admite clúster, por lo que la alta disponibilidad requeriría una licencia comercial o un esquema activo-pasivo. El costo dominante no es la nube sino el campo: los planes de datos y el hardware representan más del 99 % del total a gran escala, y la nube cuesta unos $ 50 por marquesina al mes. La banda muerta es lo que hace viable la escala: sin ella, 2.000 marquesinas generarían 518 millones de filas al mes (62 GB sin comprimir), más de lo que admite el disco de 100 GB del plan actual en pocos meses.

Conclusiones, Limitaciones y Trabajo Futuro

La solución cumple los ocho requisitos del proyecto de extremo a extremo y sobre infraestructura propia: los datos viajan cifrados y autenticados desde el nodo hasta la base, el fog garantiza la entrega pese a cortes de Internet, el caficultor ve su marquesina en un gemelo digital, las alertas llegan al celular y el administrador obtiene un dataset etiquetado para aprendizaje automático. La banda muerta redujo el tráfico en 88,9 % sin degradar la latencia ni la disponibilidad, y los SLI se mantienen por encima de sus objetivos.

Limitaciones. (a) Las tres marquesinas comparten un único sensor físico: una anomalía real aparece en las tres a la vez y la variabilidad entre marquesinas es simulada. (b) La calibración del LDR y del MQ-7 es aproximada (curvas típicas, sin instrumento de referencia), y con la resistencia de 20 kΩ la luz se satura cerca de 1 klx; la lectura de luz solo es válida desde el firmware 3.0.4, que corrigió un divisor montado al revés de lo supuesto. (c) La señal WiFi entre el nodo y la Raspberry fue débil a pocos metros (−80 dBm), lo que obligó a acercar los equipos. (d) Por facilidad durante el desarrollo se mantuvieron el acceso SSH con clave y el sudo sin clave en la Raspberry. (e) EMQX Community permite un solo nodo, de modo que no hay alta disponibilidad del broker.

Trabajo futuro. Autenticación mutua con certificados de cliente por nodo; endurecimiento de la Raspberry y del SSH; agregación en el borde (mínimo, promedio y máximo por minuto) como segunda técnica de optimización; implementación del OTA descrito; un sensor por marquesina real y la medición de la humedad del grano para entrenar el modelo de tiempo restante de secado con el dataset recolectado.

Referencias

Banks, A., Briggs, E., Borgendale, K., & Gupta, R. (Eds.). (2019). *MQTT version 5.0* (OASIS Standard). OASIS. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

Barker, E. (2020). *Recommendation for key management: Part 1 – General* (NIST SP 800-57 Part 1 Rev. 5). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-57pt1r5

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

EMQ Technologies. (2026a). *EMQX documentation*. Recuperado el 2 de noviembre de 2026, de https://docs.emqx.com/

EMQ Technologies. (2026b). *NanoMQ documentation*. Recuperado el 2 de noviembre de 2026, de https://nanomq.io/docs/

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. En *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. https://doi.org/10.17487/RFC8446

Roa, G., Oliveros, C. E., Álvarez, J., Ramírez, C. A., Sanz, J. R., Dávila, M. T., Álvarez, J. R., Zambrano, D. A., Puerta, G. I., & Rodríguez, N. (1999). *Beneficio ecológico del café*. Cenicafé.

Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal, 3*(5), 637–646. https://doi.org/10.1109/JIOT.2016.2579198

Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.

Timescale. (2026). *TimescaleDB documentation*. Recuperado el 2 de noviembre de 2026, de https://docs.timescale.com/

Yousefpour, A., Fung, C., Nguyen, T., Kadiyala, K., Jalali, F., Niakanlahiji, A., Kong, J., & Jue, J. P. (2019). All one needs to know about fog computing and related edge computing paradigms: A complete survey. *Journal of Systems Architecture, 98*, 289–330. https://doi.org/10.1016/j.sysarc.2019.02.009

Enlaces de la entrega

Repositorio: https://github.com/Robindussan/a3-cafe-cloud-iot · Panel: https://panel.agrotecnet.cloud · Video de demostración: (se agrega al publicar el video)